

Lecture #11: Best practice for scaling LLM training

- scaling law = smaller model / more data
- Does Chinchilla's approach to scaling actually work?
 - Can we save compute when training and fitting these things?
 - Should we pick particular architectures / parameterizations to scale nicely?

Cerebras - GPT	MinicPM	Deepseek
⇒ 0.1-13B models ⇒ trained with chinchilla recipe ⇒ more predictable scaling from mup params ⇒ mup parameterization combined with aggressive scaling for hyperparameters ⇒ chinchilla: small model + more data = good accuracy ⇒ scaling curve fits are good Development	⇒ small, high-perf LM from Tsinghua group ⇒ 1.5B model, beats most 2B/7B model ⇒ careful, extensive scaling computations + mup to stabilize train ⇒ opt mup for initialization fix aspect ratio, scale model ⇒ optimal batch size ⇒ batch size ↑ loss size ↓ ⇒ optimal/stable LR ⇒ WSD learning rate split LR into 3 phases: warmup, stable, decay LIAMA ↓ ⇒ optimal model-data ratio	⇒ 1B/67B param models ⇒ scale LLM with longterm ⇒ no any mup, directly estimate optimal batch/LR ⇒ pick WSD for training LLM ⇒ WSD warm up step-wise decay ⇒ WSD replaces replaces the typical cosine learning rate decay with a step-wise-decay learning rate WSD chinchilla

Kaplan's scaling law → chinchilla's optimal scaling law

performance scale with size/compute / data param ratio → mup stabilize LR / gradient